is more of a judgement call: you could choose to remove it, but Builder decided not to.

Now that you know about Blank Slates, you can finally fix the bug in the Computer class.

Fixing the Computer Class

To turn Computer into a *Blank Slate (66)* and fix the display method bug, you and Bill make it a subclass of BasicObject:

```
class Computer < BasicObject
  # ...
```

There is one last improvement you can make to this class. BasicObject doesn't have a respond_to? method. (respond_to? is a method of BasicObject's subclass Object.) Because you don't have respond_to?, you can delete the now pointless respond_to_missing? method that you and Bill added back in *respond_to_missing?, on page 62*. Once you do that, you're finally done with the method_missing-based implementation of Computer.

Wrap-Up

Let's review today's work. You and Bill started with a Computer class that contained lots of duplication. (The original class is in *Double, Treble... Trouble, on page 47*.) You removed the duplication in two different ways.

Your first attempt relied on *Dynamic Methods (51)* and *Dynamic Dispatch (49)*:

methods/computer/more_dynamic_methods.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
    data_source.methods.grep(/^get_(.*)_info$/) { Computer.define_component $1 }
  end

  def self.define_component(name)
    define_method(name) do
      info = @data_source.send "get_#{name}_info", @id
      price = @data_source.send "get_#{name}_price", @id
      result = "#{name.capitalize}: #{info} ($#{price})"
      return "* #{result}" if price >= 100
      result
    end
  end
end
```